DLBAIPCV01 · PROJECT: COMPUTER VISION

Handwritten Digit Recognition

A comparison of Chollet's Keras CNN and a compact approximation of Le Cun et al. (1989)

Project report

Author: Mubeen Ahmed Soomro

Dataset: MNIST handwritten digits

Prepared: 31 October 2026

Matriculation No.: 102303212

The comparison is descriptive. Each network was trained once under a shared protocol. The historical network is approximated, not reproduced.

Contents

List of figures and tables 2
List of abbreviations 2
Abstract 3
1. Introduction: objective and preparations 3
2. Dataset 4
3. Implementation 4
4. Results and error analysis 6
5. Resources, process, and reflection 8
6. Conclusion 9
References 9
Appendix A. Experiment code 10

List of figures and tables

Figure 1. Shared training and evaluation pipeline.
Figure 2. Layer sequence of the two Keras networks.
Figure 3. Held-out test accuracy.
Table 1. Original specification and the two Keras settings.
Table 2. Test loss, test accuracy, parameter count, and fit time.
Table 3. Per-class recall on the 10,000-image test split.
Table 4. Resources used in the recorded run.

List of abbreviations

Abbreviation	Meaning
Adam	Adaptive moment estimation optimiser
CNN	Convolutional neural network
MNIST	Modified National Institute of Standards and Technology digits
ReLU	Rectified linear unit

Abstract

This project compares two convolutional networks on the MNIST digit set. The first is the Keras example published by Chollet (2021): two convolution stages with 32 and 64 filters, max pooling, 50% dropout, and a ten-class softmax. The second is a compact network arranged like the 1989 architecture of Le Cun et al.: four 5×5 feature maps, average pooling, twelve 5×5 maps, a further average-pooling stage, and the same ten-class output. A 25% dropout layer stands in for the original sparse connections between the second and third hidden stages. Both networks use the same MNIST split, pixel scaling, Adam optimiser, batch size of 128, 15 epochs, and a 10% validation split.

On the 10,000-image test set, Chollet's network reached 99.06% accuracy and a cross-entropy loss of 0.02757, with 34,826 trainable parameters and a fit time of 166.4 seconds. The compact network reached 98.32% accuracy and a loss of 0.05131, with 3,246 parameters and a fit time of 58.5 seconds. The smaller network therefore used 90.7% fewer parameters and finished fitting sooner, at a cost of 0.74 percentage points of accuracy on this run. Each model was trained once. The figures support a description of that run. They do not support a claim of statistical significance, and the compact network is not an exact copy of the 1989 system.

Graphical abstract

Input	Models	Shared protocol	Held-out result
MNIST 28×28 grayscale 60,000 train / 10,000 test	Chollet baseline Le Cun-inspired compact model	Adam, batch 128 15 epochs 10% validation	99.06% vs 98.32% 34,826 vs 3,246 parameters 166.4 s vs 58.5 s

Figure element. The four cells summarise input, models, protocol, and the single recorded comparison.

1. Introduction: objective and preparations

1.1 Situation and task

Handwritten digit recognition is a small computer-vision problem with a clear output: each image must be assigned to one of ten labels. Convolutional networks suit that problem because a digit can appear in different positions and with different stroke widths. Shared filters reuse the same local detector across the image, and subsampling reduces the spatial size before classification. Le Cun et al. (1989, pp. 399–400) treat local receptive fields and weight sharing as a way to put prior knowledge about shape into the network and to cut the number of free parameters.

The practical task is narrower than a general study of digit recognition. Chollet (2021) publishes a short Keras network for MNIST. Le Cun et al. (1989) describe an earlier network for postal digits, including a fixed pattern of connections between hidden stages H2 and H3. The task is to modify the Keras example so that the resulting network resembles that historical design, to state every change, and to compare the two Keras networks on the same data and the same training schedule.

1.2 Objective

The objective is a controlled comparison, not a claim that one architecture is universally better. The comparison covers test accuracy, test cross-entropy, trainable parameter count, fit time, and the distribution of class-level errors. A second objective is to keep the historical claim honest. Differences in dataset, optimiser, and connectivity mean that the 1989 test error cannot be placed beside the MNIST scores as if the experiments were the same.

1.3 Planned procedure

The plan had four stages. First, the MNIST split and the two source descriptions were fixed, so that later changes would be visible against a written baseline. Second, both networks were implemented in one script and trained with the same preprocessing, optimiser, batch size, epoch count, validation split, and random seed. Third, test loss, test accuracy, parameter counts, fit time, learning curves, and confusion matrices were saved from that run. Fourth, the compact network was interpreted as an approximation, and the places where it departs from Le Cun et al. (1989) were written down before the accuracy numbers were treated as a ranking.

Adam and a fixed epoch budget were chosen for both models so that the comparison would isolate architecture rather than a search over learning rates. Dropout was chosen as the modern substitute for the sparse H2-to-H3 table because the task statement allows that substitution. No hyperparameter search was planned. A single seed was planned for the recorded run, with the limitation stated in the results rather than hidden.

1.4 Preparatory work

Preparatory reading covered the 1989 paper, the Keras MNIST example, and the Keras description of the dataset. The feature-map counts used here follow that paper's account of four maps in the first convolution stage and twelve in the second, with local averaging between them (Le Cun et al., 1989, pp. 400–402). The Keras example supplied the reference settings: 3×3 convolutions, 32 then 64 filters, max pooling, 50% dropout, softmax, Adam, batch size 128, and 15 epochs (Chollet, 2021). No extra labelled data were collected.

1.5 Structure of the report

Section 2 describes MNIST. Section 3 records the training protocol and the two architectures, including the parameter table required by the task. Section 4 reports the held-out metrics and the class-level errors. Section 5 covers resources, the sequence of work, and the limits of the result. Section 6 states the conclusion. The experiment code is placed in Appendix A.

2. Dataset

MNIST contains greyscale images of handwritten digits, each stored as a 28×28 array with pixel values from 0 to 255. The Keras distribution used in this project provides a fixed partition of 60,000 training images and 10,000 test images, with labels 0–9 (Keras, n.d.). The digits are size-normalised and centred in the frame. The set is derived from the National Institute of Standards and Technology special databases and was released as a common benchmark for digit classifiers (LeCun, Cortes, & Burges, n.d.).

Class counts are close but not identical. That matters for reading per-class recall, because a rare digit is not the source of the gaps reported later; every class has on the order of a thousand test images. The test split was not used to choose the number of epochs, the optimiser, or the dropout rate. It was used once, after training, for the numbers in Section 4.

The 1989 experiment did not use this set. It used digits taken from United States postal ZIP codes, including a mix of handwritten and machine-printed numerals, and it reported error on that postal test material (Le Cun et al., 1989, pp. 397, 402). MNIST is the right dataset for a comparison with Chollet's example. It is the wrong dataset for a numerical claim about the original postal result.

3. Implementation

3.1 Preprocessing and shared protocol

Training and test images were cast to 32-bit floating point and divided by 255, so intensities lie in [0, 1]. A channel axis was added, giving tensors of shape 28×28×1. Labels were converted to ten-way one-hot vectors because both models are trained with categorical cross-entropy. Keras `validation_split=0.1` holds out the last 10% of the training array for validation. With 60,000 training images, that is 54,000 images for weight updates and 6,000 for the curves. The split is deterministic given the stored order of the array. It is not a stratified redraw.

Both models use Adam (Kingma & Ba, 2015) at the library default settings, a batch size of 128, 15 epochs, and shuffling inside `fit`. The random seed is 2026. TensorFlow's deterministic-operation flag is enabled where the installed build accepts it. Fit time is the wall-clock duration of `model.fit` only. The later test evaluation is excluded, so the times do not include a second full pass over the test set beyond what `fit` itself does. The recorded run used TensorFlow 2.18.0 and Python 3.12.14 on a Windows Intel CPU.

The shared path is short. MNIST is loaded, scaled, fit with the validation split, and then scored on the 10,000 test images. Figure 1 shows that path. The same path is used for both networks, which is the point of the design: differences in the table in Section 4 are not differences in the data loader.

Load	Scale	Fit	Score
MNIST 60,000 / 10,000	float32 / 255 shape 28×28×1	54,000 train 6,000 validation	10,000 test images loss and accuracy

Figure 1. Shared data path. The test split is not used to stop training or to pick a model.

3.2 The two networks

Chollet's network is implemented as published. A 3×3 convolution with 32 filters and ReLU is followed by 2×2 max pooling. A second 3×3 convolution with 64 filters and ReLU is followed by another 2×2 max pool. The volume is flattened, half of the units are dropped, and a dense layer with softmax produces ten class probabilities (Chollet, 2021). Valid padding and the default stride of one are left unchanged.

The compact network follows the stage order in Le Cun et al. (1989, pp. 400–402). The first convolution, called H1 here, has four feature maps and 5×5 kernels. The activation is tanh, which is closer to the smooth squashing functions of that period than ReLU is. A 2×2 average pool forms H2 and corresponds to the paper's local averaging and 2:1 subsampling. The next convolution, H3, has twelve 5×5 maps, again with tanh. A 2×2 average pool forms H4. A dense softmax layer maps the flattened H4 volume to ten classes.

The original H2-to-H3 pattern is not a full connection from every map to every next map. The paper specifies which maps are joined (Le Cun et al., 1989, Table 1). Building that mask in Keras is possible, but it is not what was implemented. A dropout layer with rate 0.25 is used instead, as the task explicitly allows. Dropout randomly zeroes activations during training and rescales the remaining path at test time (Srivastava et al., 2014). In this network the layer sits after the H3 convolution and before H4 pooling. That placement regularises the twelve-map representation. It does not delete a fixed subset of incoming connections. The substitution is therefore a regularisation proxy, not a structural copy of the 1989 table.

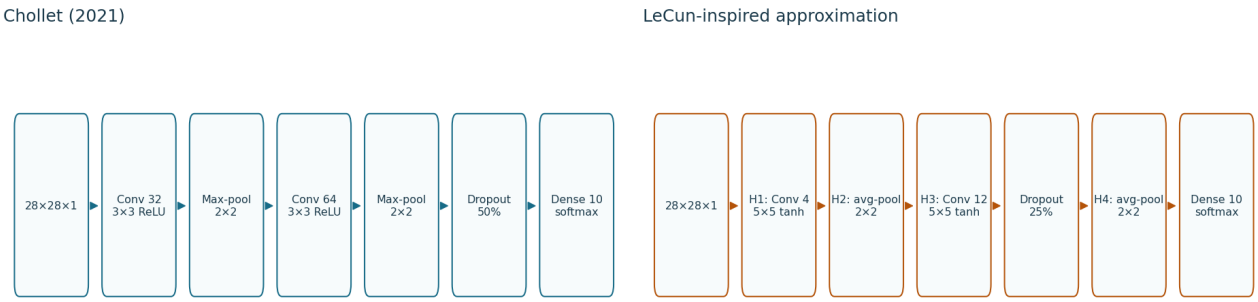


Figure 2. Layer order actually trained. Pooling and dropout add no trainable weights. Kernel sizes and map counts differ on purpose.

3.3 Parameter table and the changes that were accepted

Table 1 places the original description beside Chollet's example and beside the network that was trained. The third column is the implementation under test. Where it differs from the first column, the difference is a design choice, not an unnoticed drift.

Setting	Le Cun et al. (1989)	Chollet (2021)	This implementation
Dataset	Postal ZIP digits, not MNIST	MNIST	MNIST
Input	Normalised digit plane	28×28×1, scaled to [0, 1]	28×28×1, scaled to [0, 1]
Partition	Paper's own postal split	60,000 / 10,000; 10% of train held out	Same Keras split and 10% holdout
Feature maps	H1: 4 maps, 5×5; H3: 12 maps, 5×5	32 then 64 filters, 3×3	4 then 12 filters, 5×5
Activation	Smooth squashing units	ReLU	tanh on both convolutions
Subsampling	H2 and H4, local average, 2:1	Two 2×2 max pools	Two 2×2 average pools
H2 to H3	Sparse fixed connection table	All channels fully convolved	All channels fully convolved; 25% dropout as proxy
Classifier	H4 connected to 10 outputs	Flatten, dropout 50%, dense 10, softmax	Flatten, dense 10, softmax
Optimiser	Second-order back-propagation; about 30 passes reported	Adam, batch 128, 15 epochs	Adam, batch 128, 15 epochs

Setting	Le Cun et al. (1989)	Chollet (2021)	This implementation
Loss	Historical error criterion	Categorical cross-entropy	Categorical cross-entropy
Trainable weights	2,578 reported	34,826 measured	3,246 measured

Table 1. Settings taken from the 1989 paper, from Chollet's example, and from the code in Appendix A. The shaded column is the network that was trained as the historical approximation.

The measured sizes can be checked by hand. In Chollet's network the first convolution has 3×3×1×32 weights plus 32 biases, which is 320. The second has 3×3×32×64 weights plus 64 biases, which is 18,496. After two valid 3×3 convolutions and two 2×2 pools the map is 5×5×64. The dense layer then has 1,600×10 weights plus 10 biases, which is 16,010. The sum is 34,826. Pooling, flatten, and dropout contribute none.

In the compact network the first convolution has 5×5×1×4 weights plus 4 biases, which is 104. The second has 5×5×4×12 weights plus 12 biases, which is 1,212. Average pooling with stride 2 leaves a 4×4×12 volume, so the classifier has 192×10 weights plus 10 biases, which is 1,930. The sum is 3,246.

That count is higher than the 2,578 independent parameters reported for the original model. The gap is expected and should not be rounded away. Keras average pooling has no trainable gain on the pooled cell, but the convolution from H2 into H3 is dense: every one of the four maps feeds every one of the twelve filters. The 1989 table does not do that. A sparse mask would remove a large part of the 1,200 kernel weights in H3. Bias terms and the exact spatial size of the classifier add a further, smaller difference. The 3,246 figure is the honest size of the network that was trained. It is not a claim that the 1989 parameter count was reproduced.

Four other changes are accepted for the same reason. The data are MNIST rather than postal digits. The optimiser is Adam for 15 epochs rather than the paper's second-order procedure. The loss is categorical cross-entropy. The output non-linearity is softmax. These choices make the two Keras models comparable to each other. They also mean that a reader should not subtract the 1989 postal error from 1.68% and call the difference an improvement.

4. Results and error analysis

4.1 Held-out metrics

Each network was trained once, with seed 2026. Table 2 gives the test evaluation after the fifteenth epoch. Correct counts are accuracy multiplied by 10,000 and rounded to the nearest image.

Model	Parameters	Fit time (s)	Test loss	Test accuracy	Correct / 10,000
Chollet (2021)	34,826	166.4	0.02757	99.06%	9,906
Le Cun approximation	3,246	58.5	0.05131	98.32%	9,832

Table 2. Single-run comparison on the Keras test split. Fit time is wall-clock time of model.fit on the recorded CPU.

Chollet's network is ahead by 0.74 percentage points, which is 74 additional correct images out of 10,000, and its cross-entropy is lower (0.02757 against 0.05131). The compact network uses 3,246 / 34,826 = 9.3% of the parameter count, a reduction of 90.7%, and its fit finished in 58.5 seconds against 166.4 seconds. On this machine the smaller model was about 2.8 times faster to fit. Those times include the cost of the layers as implemented in TensorFlow 2.18. They are not a hardware benchmark and will move if the same script is run on another CPU or with another build.

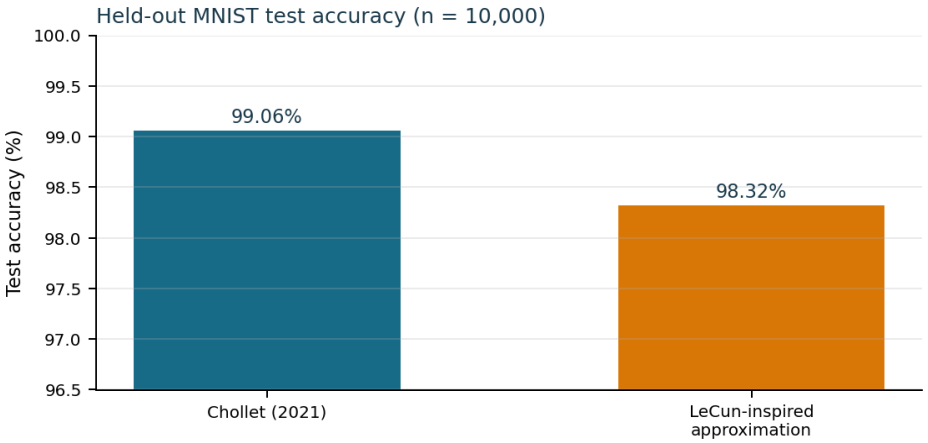


Figure 3. Test accuracy. The vertical axis starts at 96.5% so that the gap remains visible. Both results sit well above that floor.

Validation accuracy for both networks rose quickly and then flattened. Validation loss fell over the 15 epochs without a late rise that would indicate obvious overfitting on the 6,000 validation images. The curves are stored with the experiment output and are not treated as a model-selection tool: training was not stopped early, and the test set was not inspected in order to pick an epoch. Because the curves come from one seed, a crossing or a small gap between them should not be read as a stable ranking of learning speed.

4.2 Where the remaining errors sit

Errors are uncommon for both classifiers. The largest single off-diagonal cell for Chollet's network was a true 6 predicted as 0, with 8 images. For the compact network the largest cell was a true 7 predicted as 2, with 13 images. True 9 predicted as 7, and true 9 predicted as 4, each occurred 8 times. These are counts from one test pass. They are not a theory of why those shapes collide.

Digit	Chollet recall	Compact-model recall
0	99.80%	99.39%
1	99.91%	99.30%
2	98.84%	98.45%
3	99.31%	98.22%
4	99.39%	98.88%
5	99.33%	98.65%
6	98.12%	98.33%
7	98.93%	97.67%
8	98.67%	97.84%
9	98.22%	96.43%

Table 3. Recall on the test split, by true label. Recall is the share of images of that digit that were classified correctly.

Chollet's lowest recall is digit 6 (98.12%), then digit 9 (98.22%). The compact network's lowest recall is digit 9 (96.43%), then digit 7 (97.67%). Digit 6 is the one class on which the compact network is slightly higher. The pattern is consistent with the overall gap rather than with a collapse on one digit: nine of the ten classes stay above 97% for the smaller model, and all ten stay above 98% for Chollet's model. The practical difference is 74 images, concentrated but not confined to 7 and 9.

The confusion matrices themselves are produced by the script in Appendix A and saved as an image next to the metrics file. They are not repeated as a full numeric grid here, because a 10×10 grid of small counts is harder to check in print than the recall table and the three dominant confusions named above.

4.3 What the comparison does and does not show

If the goal on this benchmark is test accuracy under the shared protocol, Chollet's network is the better of the two runs. If the goal is a small parameter budget or a shorter fit on this CPU, the compact network is the better of the two runs, and the accuracy cost is under one percentage point. Memory during inference, latency per image, and

the operational cost of a 7-versus-2 error were not measured, so those criteria stay outside the result.

The comparison does not show that 99.06% would beat 98.32% on every seed. Dropout and weight initialisation both move between runs. A mean and a standard deviation over several seeds would be required before the 0.74 point gap could be treated as a stable difference. The comparison also does not show that the compact network matches Le Cun et al. (1989) on postal mail. That experiment used other images, another optimiser, and a sparse connection table that this code does not implement.

5. Resources, process, and reflection

5.1 Resources

The project did not need paid data or a cloud budget. MNIST is distributed with Keras. Training both networks for 15 epochs was feasible on a laptop CPU, which is a direct consequence of the input size and of the decision not to search hyperparameters. Table 4 lists what was actually used. No labour-hour log was kept; the constraint that mattered was machine time for a full rerun, which is a few minutes rather than a multi-day job.

Resource	Use in this project
MNIST via Keras	Fixed 60,000 / 10,000 split; no extra labels collected
Python 3.12.14	Experiment script and figure export
TensorFlow / Keras 2.18.0	Both model definitions, training, and evaluation
Windows Intel CPU	The fit times in Table 2 were measured here
Seed 2026	Initialisation and dropout masks for the recorded run

Table 4. Resources for the recorded run. A different CPU will change the times and can change the last digits of the metrics.

5.2 Process

The work followed the four stages set out in Section 1.3. The source settings were written into Table 1 before the test numbers were interpreted, so that a tempting reading — treating the higher accuracy as proof that the historical design had been beaten on its own terms — would have to pass that table first. It does not pass it. The implementation stage then kept the data loader, the loss, the optimiser, the batch size, and the epoch count in one place, used by both builders. That is why a difference in Table 2 can be traced to the layer lists in Figure 2 rather than to an accidental change of protocol.

One difficulty showed up as soon as the parameter counts were printed. A network that is described as “the 1989 model” but contains 3,246 weights cannot be left next to a paper that reports 2,578 without a sentence on the difference. Section 3.3 is that sentence. A second difficulty is dropout’s position. Putting it after H3 matches the need for a Keras regulariser, and the task allows the substitution, but it does not reproduce Table 1 of the paper. Leaving the limitation in the method section, rather than only in a final paragraph, keeps the result from being quoted without the caveat.

The run itself completed without a failed epoch or a divergent loss. Validation loss declined across the schedule. No mid-run change of learning rate or of architecture was made. The absence of a search is part of the method: the comparison is conditional on these settings.

5.3 Reflection against the source models

Le Cun et al. (1989) argue that weight sharing and local averaging encode assumptions about handwriting and reduce the number of free parameters. The compact network takes that argument literally at the level of map counts and pooling type, and the measured size, 3,246 against 34,826, shows what that choice still buys even after the connection table has been simplified. The accuracy gap shows the other side of the same argument. A larger set of 3×3 detectors, with max pooling and a heavier dropout rate, fits this particular benchmark more tightly. Capacity is not free, but on MNIST, with 54,000 training images, the extra capacity did not produce a visible validation-loss rebound inside 15 epochs.

Chollet’s example is useful here as an engineering baseline rather than as a theory of vision. It fixes a public, short specification so that a modified network has something definite to be compared with. Matching that specification exactly, including ReLU and max pooling, avoids a second, unplanned experiment in which the “reference” has

also been edited.

Dropout theory does not say that a random mask equals a hand-designed sparse graph. Srivastava et al. (2014) present dropout as noise on hidden units during training. The 1989 table is a permanent structural constraint. Using one in place of the other is reasonable for a modern reimplementation, and it is the substitution the task names, but the two mechanisms will not select the same features. A later version could apply a fixed binary mask on the H3 kernels and drop the dropout layer, which would separate those two effects.

The result is strong enough to document and too thin to generalise. One seed, one machine, and one epoch budget support a project record. They do not support a statement about expected accuracy for the next training run, and they do not support a deployment choice in which a 7 read as a 2 would be costly. For any further use, the minimum addition is repeated seeds and a report of mean and spread. A closer historical implementation would add the sparse mask and, only then, a comparison that speaks to connectivity rather than to dropout.

5.4 What should change in a second iteration

Three changes would improve the record without changing the question. First, the script should be run for several seeds, and Table 2 should gain a mean and a standard deviation. Second, the H2-to-H3 mask should be coded directly, with dropout removed, so that the historical constraint is no longer conflated with regularisation. Third, the test-set confusion matrices should be exported at a size that remains readable in print; the recall table already carries the class summary, but the full grid belongs in the appendix once it can be checked by eye. None of these changes is required to answer the present task. Each of them closes a limit that Section 4.3 states openly.

6. Conclusion

The project produced a working comparison between Chollet's published MNIST network and a compact Keras network built on the stage order of Le Cun et al. (1989). Under one protocol — the same split, the same scaling, Adam, batch size 128, 15 epochs, and seed 2026 — the reference network reached 99.06% test accuracy and a loss of 0.02757. The compact network reached 98.32% and a loss of 0.05131. It did so with 3,246 parameters rather than 34,826, and with a fit time of 58.5 seconds rather than 166.4 seconds on the recorded CPU.

The objective is met as a descriptive comparison. It is not met as a reproduction of the 1989 system, and the report does not claim that outcome. The data are MNIST rather than postal digits, the optimiser is Adam rather than the paper's second-order method, and the sparse connection table is replaced by dense convolution plus 25% dropout. The parameter gap between 3,246 and the reported 2,578 follows from that dense connection and from Keras accounting of biases. Those points are part of the result, not footnotes to it.

For later study the useful next step is repetition across seeds, then a masked H3 convolution that can be compared with the dropout proxy. For professional work the same record is a reminder that a public baseline and a written list of departures are worth more than a single accuracy figure. A model that is one point less accurate and an order of magnitude smaller can be the right engineering choice, but only after the missing runs, and only where the remaining errors — here, mostly 7 and 9 on the compact network — are acceptable.

References

Chollet, F. (2021). *Simple MNIST convnet*. Keras. https://keras.io/examples/vision/mnist_convnet/

Keras. (n.d.). *MNIST digits classification dataset*. https://keras.io/api/datasets/mnist/

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980

Le Cun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Handwritten digit recognition with a back-propagation network. In *Advances in Neural Information Processing Systems 2* (pp. 396–404). Morgan Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1989/file/53c3bce66e43be4f209556518c2fcb54-Paper.pdf

LeCun, Y., Cortes, C., & Burges, C. J. C. (n.d.). *The MNIST database of handwritten digits*. http://yann.lecun.com/exdb/mnist/

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*, 1929–1958.

List of appendices

Appendix A. Experiment code (the script that produces Table 2, the recall figures, and the saved confusion matrices).

Appendix A. Python source code

This listing matches the accompanying project_cv.py file.

```
"""MNIST CNN comparison for DLBAIPCV01 Project: Computer Vision.

Models: Chollet's published Simple MNIST ConvNet and a modern Keras approximation
of the LeCun et al. (1989) convolutional architecture.
"""
from __future__ import annotations

import json
import os
import platform
import time
from pathlib import Path

os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
os.environ.setdefault("TF_ENABLE_ONEDNN_OPTS", "1")

import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

ROOT = Path(__file__).resolve().parent
RESULTS_DIR = ROOT / "results"
FIGURES_DIR = ROOT / "figures"
SEED = 2026
EPOCHS = 15
BATCH_SIZE = 128


def load_mnist():
    """Load the canonical 60k/10k MNIST split and apply shared preprocessing."""
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_train = (x_train.astype("float32") / 255.0)[..., np.newaxis]
    x_test = (x_test.astype("float32") / 255.0)[..., np.newaxis]
    y_train_oh = keras.utils.to_categorical(y_train, 10)
    y_test_oh = keras.utils.to_categorical(y_test, 10)
    return (x_train, y_train, y_train_oh), (x_test, y_test, y_test_oh)


def build_chollet_model():
    """Chollet (2021) Simple MNIST ConvNet, reproduced from the published example."""
    model = keras.Sequential(
        [
            keras.Input(shape=(28, 28, 1), name="digit_image"),
            layers.Conv2D(32, kernel_size=(3, 3), activation="relu", name="conv_1"),
            layers.MaxPooling2D(pool_size=(2, 2), name="max_pool_1"),
            layers.Conv2D(64, kernel_size=(3, 3), activation="relu", name="conv_2"),
            layers.MaxPooling2D(pool_size=(2, 2), name="max_pool_2"),
            layers.Flatten(name="flatten"),
            layers.Dropout(0.5, seed=SEED, name="dropout"),
            layers.Dense(10, activation="softmax", name="class_probabilities"),
        ],
        name="chollet_simple_mnist_convnet",
    )
    model.compile(
        loss="categorical_crossentropy",
        optimizer=keras.optimizers.Adam(),
        metrics=["accuracy"],
    )
    return model


def build_lecun_approximation():
    """LeCun-inspired feature extraction and subsampling, implemented in Keras.

    H1: four 5x5 shared-weight feature maps. H2: 2x2 local averaging/subsampling.
    H3: twelve 5x5 feature maps. Dropout is an explicit modern regularization
    proxy for the historical H2-to-H3 sparse connectivity; it is not an exact
    structural reproduction of the original connection table. H4: subsampling.
    """
    model = keras.Sequential(
        [
            keras.Input(shape=(28, 28, 1), name="digit_image"),
            layers.Conv2D(4, kernel_size=(5, 5), activation="tanh", name="H1_features"),
            layers.AveragePooling2D(pool_size=(2, 2), name="H2_subsampling"),
            layers.Conv2D(12, kernel_size=(5, 5), activation="tanh", name="H3_features"),
```

Appendix A. Python source code (continued)

```
            layers.Dropout(0.25, seed=SEED, name="H2_H3_connectivity_proxy"),
            layers.AveragePooling2D(pool_size=(2, 2), name="H4_subsampling"),
            layers.Flatten(name="flatten"),
            layers.Dense(10, activation="softmax", name="class_probabilities"),
        ],
        name="lecun_1989_modern_approximation",
    )
    model.compile(
        loss="categorical_crossentropy",
        optimizer=keras.optimizers.Adam(),
        metrics=["accuracy"],
    )
    return model


def _run_one(name, model, x_train, y_train_oh, x_test, y_test_oh, y_test):
    start = time.perf_counter()
    history = model.fit(
        x_train,
        y_train_oh,
        batch_size=BATCH_SIZE,
        epochs=EPOCHS,
        validation_split=0.1,
        shuffle=True,
        verbose=2,
    )
    training_seconds = time.perf_counter() - start
    test_loss, test_accuracy = model.evaluate(x_test, y_test_oh, verbose=0)
    probabilities = model.predict(x_test, batch_size=512, verbose=0)
    predictions = probabilities.argmax(axis=1)
    cm = tf.math.confusion_matrix(y_test, predictions, num_classes=10).numpy()
    return {
        "name": name,
        "parameters": int(model.count_params()),
        "training_seconds": float(training_seconds),
        "test_loss": float(test_loss),
        "test_accuracy": float(test_accuracy),
        "history": {k: [float(v) for v in vals] for k, vals in history.history.items()},
        "confusion_matrix": cm.tolist(),
        "predictions": predictions.tolist(),
    }


def run_experiment():
    """Train both models once and save the machine-readable measurements/plots."""
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)
    keras.utils.set_random_seed(SEED)
    try:
        tf.config.experimental.enable_op_determinism()
    except (AttributeError, RuntimeError):
        pass

    (x_train, y_train, y_train_oh), (x_test, y_test, y_test_oh) = load_mnist()
    print(f"Dataset: train={x_train.shape}, test={x_test.shape}; seed={SEED}")
    print(
        f"Runtime: TensorFlow {tf.__version__}; Python {platform.python_version()}; "
        f"{platform.processor() or platform.machine()}"
    )

    models = [
        ("Chollet (2021)", build_chollet_model()),
        ("LeCun et al. (1989), modern approximation", build_lecun_approximation()),
    ]
    model_results = []
    for name, model in models:
        print(f"\nTraining {name} ({model.count_params():,} parameters)")
        model_results.append(_run_one(name, model, x_train, y_train_oh, x_test, y_test_oh, y_test))

    payload = {
        "experiment": "MNIST CNN comparison",
        "seed": SEED,
        "epochs": EPOCHS,
        "batch_size": BATCH_SIZE,
        "validation_split": 0.1,
        "training_samples": int(len(x_train)),
```

Appendix A. Python source code (continued)

```
        "validation_samples": int(len(x_train) * 0.1),
        "test_samples": int(len(x_test)),
        "tensorflow_version": tf.__version__,
        "python_version": platform.python_version(),
        "platform": platform.platform(),
        "models": model_results,
    }
    result_path = RESULTS_DIR / "experiment_results.json"
    result_path.write_text(json.dumps(payload, indent=2), encoding="utf-8")
    _save_figures(payload)
    _save_model_parameter_table(models)
    print("\nFinal test results")
    for item in model_results:
        print(
            f"{item['name']}: loss={item['test_loss']:.5f}, "
            f"accuracy={item['test_accuracy']:.5f}, "
            f"fit={item['training_seconds']:.1f}s, params={item['parameters']:,}"
        )
    print(f"Saved results to {result_path}")
    return payload


def _save_figures(payload):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from sklearn.metrics import ConfusionMatrixDisplay

    fig, axes = plt.subplots(1, 2, figsize=(12, 4.8), constrained_layout=True)
    for ax, item in zip(axes, payload["models"]):
        ConfusionMatrixDisplay(
            confusion_matrix=np.asarray(item["confusion_matrix"]),
            display_labels=np.arange(10),
        ).plot(ax=ax, colorbar=False, values_format="d", cmap="Blues")
        ax.set_title(item["name"], fontsize=10)
    fig.suptitle("MNIST test-set confusion matrices (n = 10,000)", fontsize=14, weight="bold")
    fig.savefig(FIGURES_DIR / "confusion_matrices.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(11, 4), constrained_layout=True)
    colors = ["#176B87", "#D97706"]
    for ax, metric, label in zip(axes, ["accuracy", "loss"], ["Accuracy", "Loss"]):
        for color, item in zip(colors, payload["models"]):
            key = f"val_{metric}"
            ax.plot(range(1, len(item["history"][metric]) + 1), item["history"][key],
                    marker="o", markersize=3, linewidth=1.8, color=color, label=item["name"])
        ax.set_xlabel("Epoch")
        ax.set_ylabel(label)
        ax.grid(alpha=0.25)
        ax.legend(frameon=False, fontsize=8)
        ax.set_title(f"Validation {label.lower()}")
    fig.suptitle("Learning curves", fontsize=14, weight="bold")
    fig.savefig(FIGURES_DIR / "learning_curves.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(6.5, 3.8), constrained_layout=True)
    names = ["Chollet (2021)", "LeCun-inspired"]
    acc = [m["test_accuracy"] * 100 for m in payload["models"]]
    bars = ax.bar(names, acc, color=colors, width=0.55)
    ax.set_ylim(max(90, min(acc) - 2), 100)
    ax.set_ylabel("Test accuracy (%)")
    ax.set_title("Held-out MNIST test accuracy", weight="bold")
    ax.grid(axis="y", alpha=0.25)
    for bar, value in zip(bars, acc):
            ax.text(
                bar.get_x() + bar.get_width() / 2,
                value + 0.1,
                f"{value:.2f}%",
                ha="center",
                fontsize=10,
            )
    fig.savefig(FIGURES_DIR / "accuracy_comparison.png", dpi=220, bbox_inches="tight")
    plt.close(fig)


def _save_model_parameter_table(models):
```

Appendix A. Python source code (continued)

```
    rows = []
    for name, model in models:
        for layer in model.layers:
            rows.append(
                {
                    "model": name,
                    "layer": layer.name,
                    "output_shape": str(layer.output.shape),
                    "parameters": int(layer.count_params()),
                }
            )
    out = RESULTS_DIR / "layer_parameters.csv"
    import csv
    with out.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["model", "layer", "output_shape", "parameters"])
        writer.writeheader()
        writer.writerows(rows)


if __name__ == "__main__":
    run_experiment()
```